RAMSES Data, the Converter, and the LaRT AMR Interface

Contents

1. Overview of RAMSES

RAMSES [1] is an adaptive mesh refinement (AMR) code for astrophysical hydrodynamics and N-body dynamics. It decomposes the computational domain into an octree hierarchy of grids, refining regions of interest to arbitrarily high resolution. The MPI decomposition uses a space-filling curve (Peano–Hilbert ordering) to distribute octs among CPU ranks.

Note: As of v2.10, LaRT no longer reads RAMSES binary files directly. RAMSES data must first be converted to the generic AMR format using `convert_ramses_to_generic.x` (Fortran) or `convert_ramses_to_generic.py` (Python). This document describes the RAMSES binary format as consumed by the converter and explains how `read_ramses_amr.f90` (now part of the converter, not LaRT.x) translates it into the flat leaf-cell arrays of the generic format.

2. Output Directory Layout

A RAMSES snapshot is stored in a directory named `output_NNNNN/` where `NNNNN` is the five-digit output number. For a run with $N_{\rm cpu}$ MPI tasks the snapshot contains the following per-CPU files:

```
output_00010/
    info_00010.txt            ! global metadata: units, cosmology, etc.
    amr_00010.out00001        ! AMR tree for CPU 1
    amr_00010.out00002        !   ...
    ...
    amr_00010.outNNNNN        ! AMR tree for CPU N
    hydro_00010.out00001      ! hydrodynamic variables for CPU 1
    hydro_00010.out00002
    ...
```

```
    hydro_00010.outNNNNN
    gravity_00010.out00001    ! gravitational potential (optional)
    part_00010.out00001       ! particle data (optional)
```

All `.out*` files are Fortran *unformatted sequential* binary files (each record bracketed by 4-byte integer record markers on little-endian systems).

3. The Info File (`info_NNNNN.txt`)

`info_NNNNN.txt` is a plain-text file containing global simulation parameters. The fields relevant to LaRT are listed below.

Key	Type	Meaning
`ncpu`	integer	Number of MPI ranks used for this snapshot
`ndim`	integer	Number of spatial dimensions (always 3 for 3-D runs)
`nlevelmax`	integer	Maximum AMR refinement level used
`boxlen`	real	Box length in *code units* (see below)
`unit_l`	real	Length unit: 1 code length = `unit_l` cm
`unit_d`	real	Density unit: 1 code density = `unit_d` g cm^{-3}
`unit_t`	real	Time unit: 1 code time = `unit_t` s
`gamma`	real	Adiabatic index γ of the EOS

The velocity unit is derived as `unit_v` = `unit_l`/`unit_t` (cm s^{-1}).

Cosmological runs. For cosmological simulations, `boxlen` is in Mpc/h and `unit_l` incorporates the scale factor a so that $L_{\rm phys}$ = `boxlen` $\times$ `unit_l` cm is the comoving box size. LaRT reads the physical box size as $L_{\rm cm}$ = `boxlen` $\times$ `unit_l`.

Example info file (non-cosmological).

```
ncpu         =           64
ndim         =            3
nlevelmax    =           12
ngridmax     =       500000
boxlen       =   1.000000000000000E+02
unit_l       =   3.085677581491367E+21   ! 1 kpc in cm
unit_d       =   6.770772291971677E-25
unit_t       =   3.085677581491367E+14
gamma        =   1.666666666666667E+00
```

In this example the box is 100 kpc on a side.

4. The AMR File (`amr_NNNNN.outCCCCC`)

Each CPU file stores the AMR tree data for that CPU's domain. The file consists of a *header section* followed by a sequence of *level blocks*, one per AMR level.

4.1 Header Records

The AMR header records are read in the following order:

Record	Variable(s)	Description
1	`ncpu`	Total number of MPI ranks
2	`ndim`	Number of dimensions
3	`nx, ny, nz`	Coarse grid size (typically $1 \times 1 \times 1$ or $2 \times 2 \times 2$)
4	`nlevelmax`	Maximum AMR level
5	`ngridmax`	Maximum number of grids (octs) per CPU
6	`nboundary`	Number of boundary domains
7	`ngrid_current`	Current number of active grids
8	`boxlen`	Box length (code units)
9–21	(various)	Cosmological parameters, time, etc. (skipped by LaRT)
22	`ngridlevel(:,:)`	Grid count array: `(ncpu, nlevelmax)`
23	(level sums)	Skipped
24–27	(boundary grids)	Only present if `nboundary > 0` (skipped)
28–33	(coarse-grid / ordering arrays)	`headf`, `ordering`, Hilbert keys, coarse `son`, `flag1`, `cpu_map`; skipped by LaRT

The ngridlevel array. `ngridlevel(j, l)` gives the number of octs belonging to CPU j at AMR level l. LaRT constructs a combined table `ngridfile(j, l)` that includes both domain and boundary grids.

4.2 Level Blocks

After the header, the file contains one block per AMR level $l = 1, \ldots,$ `nlevelmax`. Within each level, data are stored *domain by domain*: first all domains $j = 1, \ldots,$ `ncpu`, then all boundary domains $j =$ `ncpu`$+1, \ldots,$ `ncpu`$+$`nboundary`. For each domain j at level l (if `ngridfile`$(j,l) > 0$):

Record(s)	Variable	Description
1	`ind_grid(:)`	Global grid indices
2	`next(:)`	Linked-list next pointer
3	`prev(:)`	Linked-list previous pointer
4–6	`xg(:, 1:3)`	Oct center coordinates in code units
7	`father(:)`	Parent grid index
8–13	`nbor(:,1:6)`	Neighbor grid indices (6 faces)
14–21	`son(:,1:8)`	Child grid indices; = 0 if a child is a **leaf**
22–29	`cpu_map(:,1:8)`	CPU ownership per child
30–37	`ref_map(:,1:8)`	Refinement flags per child

Record counts assume `ndim`= 3, `twotondim`= 8.

4.3 Oct Structure and Leaf Identification

RAMSES groups cells into *octs*: cubes of $2^3 = 8$ cells sharing a common parent grid cell. Each oct is identified by its *grid index*. Within an oct, the eight child cells are indexed $\texttt{ind} = 1, \ldots, 8$ with the following ordering:

$$i_x = (\texttt{ind}-1) \bmod 2, \quad i_y = \lfloor (\texttt{ind}-1)/2 \rfloor \bmod 2, \quad i_z = \lfloor (\texttt{ind}-1)/4 \rfloor, \tag{1}$$

so that `ind`=1 is the $(-x,-y,-z)$ corner and `ind`=8 is the $(+x,+y,+z)$ corner.

A child cell at index `ind` in grid g at level l is a **leaf** if and only if

$$\texttt{son}(g, \texttt{ind}) = 0. \tag{2}$$

Otherwise the value of `son` is the grid index of the finer oct that occupies that child cell.

4.4 Cell Position Calculation

The center of the coarse grid of the root domain is the origin. For a non-cosmological run with `nx=ny=nz=1`:

$$\mathbf{x}_{\text{bound}} = \left(\frac{n_x}{2}, \frac{n_y}{2}, \frac{n_z}{2} \right). \tag{3}$$

The cell size at level l is $\Delta x_l = 2^{-l}$ (in units where `boxlen` = 1). The offset of child cell `ind` from the oct center is

$$\boldsymbol{\delta}_{\texttt{ind}} = \left(i_x - \tfrac{1}{2},\ i_y - \tfrac{1}{2},\ i_z - \tfrac{1}{2} \right) \times \Delta x_l. \tag{4}$$

The absolute position of child `ind` in grid g in fractional box units $[0,1]$ is

$$\tilde{x} = \frac{x_g + \delta_{\texttt{ind},x} - x_{\text{bound},x}}{n_x} + \frac{1}{2}, \tag{5}$$

and similarly for $\tilde{y}$ and $\tilde{z}$ (the $+1/2$ shifts the origin from the grid corner to the box corner). In the code:

```
xleaf = (xg(j,1) + xc(ind,1) - xbound(1)) / dble(nx) + 0.5d0
```

After reading, leaf positions are converted to physical units (cm):

$$x_{\text{phys}} = \tilde{x} \times L_{\text{box}}, \quad L_{\text{box}} = \texttt{boxlen} \times \texttt{unit\_l}. \tag{6}$$

5. The Hydro File (`hydro_NNNNN.outCCCCC`)

The hydro file stores the conserved hydrodynamic variables for every cell (leaf and internal alike). It is organized in strict lockstep with the AMR file: one block per level per domain.

5.1 Header Records

Record	Variable	Description
1	`ncpu`	Number of MPI ranks
2	`nvar`	Number of hydro variables per cell
3	`ndim`	Number of dimensions
4	`nlevelmax`	Maximum AMR level
5	`nboundary`	Number of boundary domains
6	`gamma`	Adiabatic index

5.2 Variable Ordering

Standard RAMSES hydro stores the following variables in order:

Index	Variable	Physical meaning
1	ρ	Gas mass density [code units]
2	ρv_x	x-momentum density [code units]
3	ρv_y	y-momentum density [code units]
4	ρv_z	z-momentum density [code units]
5	E	Total energy density (kinetic + thermal) [code units]
6	$Z/Z_\odot$	Metallicity (if `metal=.true.`)
7+	passive scalars	Additional tracers (simulation-dependent)

RMHD / jellyfish variant. Some RAMSES derivatives provide a plain-text `hydro_file_descriptor.txt` file inside `output_NNNNN/`. The `jellyfish/RMHD` outputs inspected for this work use:

Index	Variable	Meaning
1	ρ	Gas density
2–4	v_x, v_y, v_z	*velocity* (not momentum)
5–10	$B_{\mathrm{left/right}}$	Magnetic field components
11	P_{nt}	Non-thermal pressure
12	P_{th}	Thermal pressure
13+	passive scalars	Tracers / chemistry fields

In this case the velocity conversion omits the division by density and the temperature is reconstructed from thermal pressure rather than total energy.

Level block structure. For each level l and domain j, the hydro file contains:

1. Two header records (level number and domain number) — skipped.
2. For each cell index `ind`$= 1,\ldots,8$ and each variable `ivar`$= 1,\ldots,$`nvar`: one record of length `ngridfile`(j,l) containing the values for all grids of domain j at level l.

All records have the same element count `ngridfile`(j,l), so the loop structure is:

```
do ind = 1, twotondim     ! 8 cells per oct
  do ivar = 1, nvar       ! variables
    read(iu_hyd) var(:, ind, ivar)  ! (ngrida) values
  end do
end do
```

Precision. RAMSES writes hydro variables in double precision (`real*8`) by default. Some configurations compile with single precision (`real*4`). The LaRT reader handles both via the flag `ramses_hydro_prec` (set to 8 or 4 in `read_ramses_amr.f90`).

6. Physical Unit Conversions

6.1 Hydrogen Number Density

The mass density in code units is converted to CGS and then to hydrogen number density assuming a fully ionised gas with mean molecular weight $\mu_{\rm H} \approx 1$ (one proton mass per hydrogen nucleus):

$$n_H\ [\mathrm{cm}^{-3}] = \frac{\rho_{\rm code} \times \texttt{unit\_d}}{m_H}, \quad m_H = 1.6726 \times 10^{-24}\ \mathrm{g}. \tag{7}$$

The neutral fraction $n_{\rm HI}/n_H$ is computed separately in `grid_mod_amr.f90` using collisional ionisation equilibrium (CIE) if `par%use_cie_condition = .true.`, otherwise all hydrogen is assumed neutral.

6.2 Velocity

For standard RAMSES, variables 2–4 store momentum density ρv_α. The velocity in CGS is:

$$v_\alpha\ [\mathrm{cm\,s}^{-1}] = \frac{(\rho v_\alpha)_{\rm code}}{\rho_{\rm code}} \times \texttt{unit\_v}, \quad \texttt{unit\_v} = \frac{\texttt{unit\_l}}{\texttt{unit\_t}}. \tag{8}$$

For the RMHD variant described above, variables 2–4 are already *velocities* in code units, so LaRT instead uses

$$v_\alpha = v_{\alpha,\rm code} \times \texttt{unit\_v}. \tag{9}$$

LaRT receives velocities in km s^{-1} (divided by 10^5).

6.3 Temperature

For standard RAMSES, variable 5 is the *total energy density E* (kinetic plus thermal). The thermal energy density is:

$$e_{\rm th} = E - \frac{1}{2}\rho v^2 = E - \frac{(\rho v_x)^2 + (\rho v_y)^2 + (\rho v_z)^2}{2\rho}. \tag{10}$$

The specific thermal energy (per unit mass) in physical units is:

$$\varepsilon\ [\mathrm{erg\,g^{-1}}] = \frac{e_{\rm th}}{\rho_{\rm code}} \times \texttt{unit\_v}^2. \tag{11}$$

The temperature follows from the ideal gas law with adiabatic index γ and mean molecular weight $\bar{\mu}$ (LaRT assumes $\bar{\mu} = 1.22$ for neutral gas):

$$T\ [\mathrm{K}] = (\gamma - 1)\,\varepsilon\,\frac{\bar{\mu} m_H}{k_B}, \quad k_B = 1.381 \times 10^{-16}\ \mathrm{erg\,K^{-1}}. \tag{12}$$

A floor of $T = 10$ K is applied after conversion.

For the RMHD / jellyfish variant, the reader uses the thermal pressure `P_th` from `hydro_file_descriptor.txt` (variable 12 in the example outputs) and reconstructs the temperature directly from the ideal gas law:

$$T = \frac{P_{\rm th}}{\rho}\frac{\bar{\mu} m_H}{k_B}, \tag{13}$$

where

$$P_{\rm th}[\mathrm{erg\,cm^{-3}}] = P_{\rm th,code} \times \texttt{unit\_d} \times \texttt{unit\_v}^2. \tag{14}$$

7. The Converter Reader: Step-by-Step

The reader in `read_ramses_amr.f90` (now part of the converter executable, not `LaRT.x`) proceeds as follows:

1. **Read `info` file.** Extract `ncpu`, `unit_l`, `unit_d`, `unit_t`, `boxlen`, and γ.
2. **Detect hydro layout.** If `output_NNNNN/hydro_file_descriptor.txt` exists, parse it to determine whether variables 2–4 are velocities or momenta and whether the thermodynamic variable is total energy or thermal pressure.
3. **Count leaf cells (first pass).** `ramses_count_leaves` opens every `amr_*.outCCCCC` file, reads the `son` arrays for all levels, and counts cells where `son` = 0. This gives the total $N_{\rm leaf}$ needed to pre-allocate output arrays.
4. **Read leaf data (second pass).** `ramses_read_all_cpus` opens *both* the AMR and hydro files simultaneously for each CPU:
   - Reads AMR grids level by level, extracting oct centers `xg` and the `son` array.
   - Reads hydro variables for the same grids.
   - For each child cell with `son` = 0 (leaf): computes its physical position, n_H, T, and **3**, and appends them to the output arrays.

5. **Convert positions.** After the second pass, leaf positions (stored as fractions of `boxlen`) are multiplied by $L_{\rm cm}$ = `boxlen` × `unit_l` to give physical positions in cm. These are then passed to `amr_build_tree` as-is (with `distance2cm = 1`).

6. **Build octree and normalize.** Handled by `grid_mod_amr.f90` (see the AMR description document).

7.1 Domain Skipping

The AMR and hydro files interleave data from all $N_{\rm cpu}$ domains and boundary domains at each level. The LaRT reader only *stores* data for the domain that matches `icpu` (the loop variable); data from other domains are skipped with `skip_records`. This reproduces the original RAMSES read pattern used in post-processing tools such as PYMSES and YT.

8. File Naming Convention

File pattern	Content
`output_NNNNN/info_NNNNN.txt`	Global metadata
`output_NNNNN/amr_NNNNN.outCCCCC`	AMR tree for CPU `CCCCC`
`output_NNNNN/hydro_NNNNN.outCCCCC`	Hydro variables for CPU `CCCCC`
`output_NNNNN/gravity_NNNNN.outCCCCC`	Gravitational potential (optional)
`output_NNNNN/part_NNNNN.outCCCCC`	Particle data (optional)

where `NNNNN` is the zero-padded 5-digit snapshot index and `CCCCC` is the zero-padded 5-digit CPU rank (1-based).

9. Architecture: RAMSES → converter → LaRT.x

As of v2.10 (mirrored in v2.00), LaRT.x no longer reads RAMSES binary snapshots directly. The flag `par%amr_type='ramses'` now triggers a fatal error with a clear message pointing to the converter. A standalone converter produces a *generic AMR file* that LaRT.x then consumes:

```
RAMSES snapshot
     |
     | convert_ramses_to_generic.x    (Fortran)
     | convert_ramses_to_generic.py   (Python)
     |
     v
generic AMR file (.h5 / .fits.gz / .dat)
     |
     v
LaRT.x  (par%amr_type = 'generic')
```

Why?

- Separates a one-off conversion step (RAMSES I/O, layout parsing, unit conversion) from the production RT loop.
- Makes adding new input simulation codes (AREPO, ATHENA, etc.) a pure converter problem; LaRT.x only knows the generic format.
- Lets physics computations (CIE, dust, Case B emissivity) be done once at conversion time and stored alongside the cells, avoiding repeated recomputation per RT run.

The RAMSES reading code (`read_ramses_amr.f90`) is still part of the repository and is described in the preceding sections of this document, but it is now compiled *only* into the converter executable, never into `LaRT.x`.

10. The Generic AMR File Format

The generic AMR format is a flat table of leaf cells, one row per cell, written in HDF5, gzipped FITS, or plain text (auto-detected by file extension).

Mandatory columns (9). `x, y, z, level, nH [cm`$^{-3}$`], T [K], vx [km/s], vy [km/s], vz [km/s]`.

Optional columns (6). Detected by column name in HDF5/FITS files; allocated only if present:

Name	Units	Used by
`metallicity`	mass fraction	dust model, ion density
`xHI`	dimensionless	ionisation model `from_file`
`n_e`	cm^{-3}	emissivity computation
`n_ion`	cm^{-3}	metal-line opacity
`emissivity`	$\mathrm{cm}^{-3}\ \mathrm{s}^{-1}$	emissivity model `from_file`
`ndust`	cm^{-3}	dust model `from_file`

Header keys.

- `BOXLEN` – side length of the AMR box in code units.
- `ORIGINX/Y/Z` – lower corner of the box. Defaults to $-$`BOXLEN`$/2$ (centered convention) when absent.
- `NLEAF` – number of leaf cells (informational; LaRT uses NAXIS2 / the HDF5 row count).

Coordinate convention. LaRT honors `ORIGINX/Y/Z`: the box is built as $[\mathrm{origin}, \mathrm{origin}+\mathrm{boxlen}]$ in the same frame as the input cell positions. Both converters now write *centered* coordinates by default, i.e. they shift RAMSES native positions from $[0, \mathrm{boxlen}]$ to $[-\mathrm{boxlen}/2, +\mathrm{boxlen}/2]$ and set `ORIGINX/Y/Z` $= -\mathrm{boxlen}/2$. Corner-based files (`ORIGINX=`

0) are still readable; LaRT prints a WARNING if the data does not cover the box center, since pole traversal starts at the box center and would otherwise return $N(HI)_{\rm pole} = 0$.

Plain-text variant.

```
# comment lines start with #
BOXLEN  <box_length_in_code_units>
NLEAF   <number_of_leaf_cells>
<x> <y> <z> <level> <nH> <T> <vx> <vy> <vz>
...
```

The plain-text format does not carry an `ORIGIN` header; the box is assumed centered.

11. Shared Physics Module: `physics_amr_mod.f90`

Used by both `LaRT.x` (in `grid_mod_amr.f90`) and the converter (in `convert_ramses_to_generic.f90`), so the formulas cannot drift between pre-computation and in-LaRT fallback.

```
public :: cie_neutral_fraction_formula  ! single CIE formula (legacy)
public :: cie_neutral_fraction_table    ! Voronov+Verner table interpolation
public :: laursen09_ndust               ! dust from metallicity
public :: caseB_lya_emissivity          ! recombination + collisional
public :: electron_density_from_xHI     ! ne ~ nH*(1 - xHI)
```

CIE neutral-fraction table. Built once on first call from the Voronov (1997) collisional ionisation rate and the Verner & Ferland (1996) Case A recombination rate:

$$x_{HI}(T) = \frac{\alpha_A(T)}{\Gamma_{HI}(T) + \alpha_A(T)},$$

with $\log_{10} T \in [3.0, 8.0]$ at $\Delta \log T = 0.1$, log-linear interpolation.

Case B Lyα emissivity.

$$\varepsilon_{Ly\alpha} = P_B \, \alpha_B \, n_e \, n_{HII} + q_{coll}(T) \, n_e \, n_{HI},$$

with α_B from Hui & Gnedin (1997), P_B from Cantalupo et al. (2008), and q_{coll} from the Harley/RASCAS fit.

Laursen+09 dust.

$$n_{\rm dust} = \frac{Z}{Z_{\rm ref}} \left(n_{HI} + f_{\rm ion} \, n_{HII} \right),$$

where $Z_{\rm ref}$ and $f_{\rm ion}$ are exposed as `par%Z_ref` (default 0.0134) and `par%f_ion_dust` (default 0.01).

12. Solar-Abundance / CIE Module: `ion_data_mod.f90`

Provides $n_{\rm ion}(n_H, Z, T, \texttt{ion\_id})$ for 11 ion species using Asplund+09 solar abundances and Gnat & Sternberg (2007) CIE ion fractions:

H I, He I, C IV, N V, O VI, Na I, Ca II, Mg II, Si IV, Si II, Al II, Fe II.

Activated by `par%ion_model = 'solar_cie'` when the input file does not carry an `n_ion` column.

13. LaRT Input Parameters for the AMR Mode

```
&parameters
 par%grid_type        = 'amr'             ! or par%use_amr_grid = .true.
 par%amr_type         = 'generic'         ! 'ramses' is now a fatal error
 par%amr_file         = 'grid.h5'        ! .h5 / .fits.gz / .dat
 par%distance_unit    = 'kpc'            ! unit of leaf positions in the file
 par%source_geometry  = 'diffuse_emissivity'
 par%spectral_type    = 'voigt'
 par%core_skip        = .true.           ! per-cell RASCAS-style core-skip
 ! Extended physics (optional; defaults reproduce previous behavior):
 par%ionization_model    = 'cie_formula' ! 'cie_formula' | 'cie_table'
                                          ! 'full_neutral' | 'from_file'
 par%dust_model          = 'global_dgr'  ! 'global_dgr' | 'laursen09'
                                          ! 'from_file'
 par%emissivity_model    = 'none'        ! 'none' | 'caseB' | 'from_file'
 par%ion_model           = 'none'        ! 'none' | 'solar_cie' | 'from_file'
 par%metallicity_global  = 0.0134        ! global Z for laursen09 / solar_cie
 par%Z_ref               = 0.0134        ! reference Z for laursen09
 par%f_ion_dust          = 0.01          ! dust survival fraction in HII
/
```

Parameter	Default	Meaning / values
`par%ionization_model`	`'cie_formula'`	`'cie_formula'` (single Voronov formula), `'cie_table'` (Voronov + Verner table), `'full_neutral'` ($x_{HI} = 1$), `'from_file'` (requires `xHI` column).
`par%dust_model`	`'global_dgr'`	`'global_dgr'` (uses `par%DGR`), `'laursen09'` (metallicity-based), `'from_file'` (requires `ndust` column).
`par%emissivity_model`	`'none'`	`'none'` (no Case B diffuse source unless `emiss_file` is set), `'caseB'` (recomb + collisional, per cell), `'from_file'` (requires `emissivity` column).
`par%ion_model`	`'none'`	`'none'` (use $n_{HI} = n_H \cdot x_{HI}$), `'solar_cie'` (Asplund+09 & Gnat-Sternberg+07), `'from_file'` (requires `n_ion` column).

`par%metallicity_global`	−1	Global Z used by `dust_model='laursen09'` or `ion_model='solar_cie'` when no `metallicity` column is present; negative ⇒ unset.
`par%Z_ref`	0.0134	Reference solar metallicity for Laursen+09 dust scaling.
`par%f_ion_dust`	0.01	Survival fraction of dust in ionized gas (Laursen+09).

The defaults are chosen so legacy input files reproduce the previous behavior exactly.

Tau normalization. `par%taumax` (when set > 0) rescales `rhokap` so that the optical depth from the box center to the $+z$ boundary, traversed along the z-axis (*pole traversal*), equals the specified value. When omitted or set to zero, the data's native column density is used directly with no rescaling.

14. Per-Cell Core-Skip (RASCAS-style)

LaRT now uses a *per-cell, per-scattering* core-skip threshold rather than a single volume-averaged value. This matches the approach in RASCAS (Michel-Dansac et al. 2020) and follows Smith et al. (2015, their Eq. 35):

$$a\tau_{\rm cell} = a_{\rm V}\,\texttt{rhokap}\,\Delta\ell_{\rm face}, \qquad x_{\rm crit} = \begin{cases} (a\tau_{\rm cell})^{1/3}/5 & \text{if } a\tau_{\rm cell} > 1 \\ 0 & \text{otherwise} \end{cases}$$

where $a_{\rm V}$ is the cell's Voigt damping parameter, `rhokap` is the line-center opacity per unit length (code units), and $\Delta\ell_{\rm face}$ is the minimum distance from the current photon position to any of the cell's six faces (i.e. the radius of the largest sphere centered at the photon position that fits inside the cell).

Why this matters. The previous behavior used $x_{\rm crit}$ derived from a single volume-averaged $\langle a_{\rm V}\rangle\langle\tau\rangle$. That prescription fails for inhomogeneous boxes where the dense scattering region has $a\tau_{\rm cell} \gg \langle a\tau\rangle$. For example, in the jellyfish galaxy with $n_H \sim 6\times10^3$ cm^{-3} embedded in a sparse hot halo, the global $\langle a\tau\rangle$ is small, $x_{\rm crit}$ collapses to 0, and a photon launched into the dense core scatters indefinitely without progress. The per-cell formula gives a locally appropriate threshold (e.g. $x_{\rm crit} \sim 200$ for the densest jellyfish cells), allowing core photons to thermalize quickly to the wings.

Implementation.

- Cartesian: `car_xcrit_local` in `grid_mod_car.f90` uses `grid%xface/yface/zface` and per-cell `grid%voigt_a`, `grid%rhokap`.
- AMR: `amr_xcrit_local` in `octree_mod.f90` uses `amr_grid%cx/cy/cz/ch` and per-leaf `amr_grid%voigt_a`, `amr_grid%rhokap`.

Both helpers are called from the resonance scatter routines (`scattering_amr.f90`, `scattering_car.f90`) when `par%core_skip = .true.` and `par%core_skip_global = .false.` (the default combination). Setting

`par%core_skip_global = .true.` falls back to the old volume-averaged Seon-Kim formula and is retained for comparison / benchmarking only.

15. Bug Fixes (2026-05-21 Session)

15.1 Bug 1: point source with `icell_amr` unset

When `par%source_geometry='point'`, `photon%icell_amr` was left at 0 by `generate_photon_amr`, while every other source-geometry branch set it. Subsequent ray-tracing then indexed the leaf arrays with 0 and crashed.
Fix: call `amr_find_leaf` after writing the point-source coordinates, and stop with a clear error if the position is outside any leaf.

15.2 Bug 2: `diffuse_emissivity` alias table not populated

`setup.f90` promotes an unset `par%geometry` to `'sphere'`. In AMR mode this previously sent `diffuse_emissivity` photons through the 1D radial-profile path that uses `emiss_prof%prob_alias`, which is never populated by the AMR code. Result: `PROB_ALIAS not associated` (`random_mt.f90:2301`).
Fix: branch on `associated(emiss_prof%prob_alias)` rather than `par%geometry`. `amr_setup_emissivity` also refactored so it (a) returns early unless `source_geometry='diffuse_emissivity'`, (b) computes Case B emissivity from `caseB_lya_emissivity` rather than `rhokap`, (c) raises an explicit error if `from_file` is selected but no `emissivity` column is present, and (d) raises an error if the total cell emissivity is zero.

15.3 Bug 3: pole traversal across octree gaps

When the octree is incomplete (e.g. a spatial subset of a larger simulation), the pole traversal from box center along $+z$ may step into an internal cell whose subtree does not cover the photon position. `amr_next_leaf` returned 0, the traversal stopped, $N(HI)_{\rm pole} = 0$, and `taumax`-normalisation fell back to `tauhomo`, producing extreme cell opacities.
Fix: two new octree helpers — `amr_find_enclosing_cell` (returns the deepest cell containing a point, leaf or internal) and `amr_gap_exit` (steps through a virtual sub-cell of an internal cell). The pole traversal is rewritten as a single loop: at each step either accumulate opacity through a leaf, or step through a gap with zero opacity, until $z \geq z_{\rm max}$.

15.4 Bug 4: `ORIGINX/Y/Z` coordinate convention

`read_generic_amr.f90` previously assumed every input file was already in centered convention regardless of the FITS/HDF5 `ORIGINX/Y/Z` headers. Files written with corner-based positions (`ORIGINX`$= 0$, e.g. from the legacy jellyfish converter) ended up with an octree whose box was offset from the input data.
Fix: the reader now returns the origin recovered from the headers, defaulting to $-\text{boxlen}/2$ when the headers are missing. `grid_mod_amr.f90` calls `amr_build_tree` with box = [origin, origin + boxlen], so the octree sits in the same frame as the input cells. Setup prints both the box extent and the actual data range, and warns if the data does not cover the box center.

15.5 Bug 5: stuck photons — octree neighbor-table self-loops

Symptom. Some jellyfish photons (point source in a dense cold core surrounded by hot $T \sim 10^9$ K gas) printed "`N photons calculated`" but never reached "`Gathering Results`" — the master–slave loop waited forever for a hung rank.

Root cause (two-part), both in `octree_mod.f90`. (i) Neighbor-table self-loops at parent-face boundaries. `amr_build_neighbors` previously queried the $+x$ neighbor of `icell` at position $(c_x + h(1+10^{-8}), c_y, c_z)$ — essentially *on* `icell`'s $+x$ face. When `icell` happens to lie on the $+x$ side of its parent, $c_x + h$ equals the parent's $+x$ face exactly. `amr_find_cell_at_level` descends from the root with `x >= cx(node)` comparisons; at the parent level the query is classified as the $+x$ sub-octant of the parent, which is `icell` itself. Result: `neighbor(+x, icell) = icell` — a self-loop.

(ii) Ancestor returns from coordinate-based lookup in sparse octrees. When the descent reaches a missing child (no data exists in that octant — common in a sparse octree where leaves cover $< 2\%$ of the box, as in the jellyfish snapshot), `amr_find_cell_at_level` returns the deepest existing ancestor. At runtime `amr_next_leaf` descends that ancestor on its low-x side (which is correct topologically) and lands back in `icell` — the same effective self-loop.

Fix.

1. In `amr_build_neighbors`, query at the would-be same-level neighbor's *center*:

$$h_\mathrm{p} = 2h, \qquad \texttt{neighbor(1, icell)} = \texttt{amr\_find\_cell\_at\_level}(c_x + h_\mathrm{p}, c_y, c_z, \ldots).$$

   The center of the $+x$ same-level neighbor is unambiguously inside the $+x$ neighbor cell, regardless of refinement level, removing the parent-face ambiguity.

2. After populating the table, walk every entry and discard any `neighbor(iface, icell)` that resolves to an ancestor of `icell` (walking the `parent` chain). Such entries are set to 0 ("no neighbor / gap"). At runtime `amr_next_leaf` returns 0 and the raytrace lets the photon exit — physically correct because the gap region has no opacity contribution.

3. `amr_next_leaf` and `amr_next_leaf_or_gap` retain the `iface`-topological sub-octant descent: a `select case (iface)` fixes the `iface`-normal sub-octant bit from the face direction (so the photon's normal coordinate, which sits exactly on a face, never enters the comparison) and uses the two transverse coordinates for the other bits.

4. All seven `raytrace_*_amr` functions revert to the original face crossing $x \leftarrow x + t_\mathrm{exit} k_x$. No epsilon nudge, no iteration cap — the underlying topology is now correct, so a stuck-photon case cannot arise from this source.

A second class of stuck-photon, where photons *do* progress but spend $\gtrsim 10^7$ scatterings inside a single galaxy-core cell, is now solved by the per-cell core-skip described in the previous section rather than by any change to the raytrace.

15.6 Bug 6: converter octree-alignment for RAMSES nx>1

Symptom. Generic AMR files produced by `convert_ramses_to_generic.py` (and the corresponding Fortran `convert_ramses_to_generic.f90`) from RAMSES runs with a non-cubic base grid — jellyfish has `nx=ny=nz=3`, `levelmin=7`, `levelmax=14`, `boxlen=300` kpc — have cell centres off the octree-natural grid by $1/3$ of a cell width on the y and z axes. `AMRGrid.read` (which descends the octree by position) loses $\sim$94% of leaves to overwrite collisions during `_insert_cell`: 8.8 M FITS rows collapse to 545 k AMR leaves.

Root cause. In RAMSES the native cell size at level L is $\Delta_L = \texttt{boxlen}/(\texttt{nx}\cdot 2^L)$, not $\texttt{boxlen}/2^L$. The LaRT octree assumes $\Delta_L = \texttt{BOXLEN}/2^{L'}$, which requires $\texttt{BOXLEN}/2^{L'} = \texttt{boxlen}/(\texttt{nx}\cdot 2^L)$ for some integer L'. This holds only when `nx` is a power of 2.

Fix. Both converters now verify that all populated leaves lie in a sub-volume re-embeddable in a power-of-2-aligned box: a sub-block of size $m \times m \times m$ base cells with m the smallest power of 2 covering the populated base-cell extent. They then

1. shrink the output `BOXLEN` to that sub-block: $\texttt{BOXLEN}_{\text{out}} = m\cdot\texttt{boxlen}/\texttt{nx}$,
2. shift cell positions so the populated sub-block occupies the centered output box,
3. bump RAMSES levels by $\log_2 m$ so they become valid octree levels on the shrunk box.

For the jellyfish run the populated base cells are $(0,0,0)\ldots(1,1,1)$, so $m = 2$, $\texttt{BOXLEN}_{\text{out}} = 2\cdot 300/3 = 200$ kpc, and the level shift is +1. Generic files produced before this fix are mis-aligned and must be regenerated.

16. Python Tools (`python/AMR_grid/`)

16.1 `convert_ramses_to_generic.py`

Purpose. Convert a RAMSES snapshot to the generic AMR file format consumed by LaRT.x. Coordinates are written in centered convention.

Usage.

```
# Basic conversion (cells + density + temperature + velocity)
python convert_ramses_to_generic.py <repo_dir> <snapnum> \
       -o sim.h5 --output-unit kpc

# With on-the-fly physics columns (CIE xHI, ne, Case B emissivity, dust)
python convert_ramses_to_generic.py <repo_dir> <snapnum> \
       -o sim.h5 --output-unit kpc \
       --compute-physics --metallicity 0.0134
```

Equivalent Fortran tool: `convert_ramses_to_generic.x` (built by `make convert_ramses`).

16.2 `extract_amr_subset.py`

Carve a cubic spatial sub-region out of a generic AMR file. Cells whose centres fall inside the cube are kept; coordinates are shifted into a centered output cube.

```
# Center + size
python extract_amr_subset.py input.fits.gz \
    --center 15 15 15 --size 30 \
    -o sub.fits.gz
```

16.3 `AMR_grid.py` — in-memory AMR construction

The default origin convention is centered ($-\text{boxlen}/2$). All optional physics columns (`metallicity`, `xHI`, `n_e`, `n_ion`, `emissivity`, `ndust`) are first-class `Cell` attributes and are written to FITS/HDF5 when set on any leaf.

17. End-to-End Example: Jellyfish

`examples/jellyfish_rmhd/` contains an end-to-end example with an analysis notebook. Workflow:

```
# 1) RAMSES -> generic (full snapshot, power-of-2 aligned)
python ../../python/AMR_grid/convert_ramses_to_generic.py \
    ~/jellyfish/RMHD 91 -o jellyfish.h5 \
    --output-unit kpc --compute-physics --metallicity 0.0134

# 2) Carve out the galaxy region (4 kpc cube around the peak)
python ../../python/AMR_grid/extract_amr_subset.py \
    jellyfish.h5 --center 0 0 0 --size 4 \
    -o jellyfish_galaxy.h5

# 3) Run LaRT (point source) and (diffuse Case-B emissivity)
mpirun -np 64 ../../LaRT.x jellyfish_pt.in
mpirun -np 64 ../../LaRT.x jellyfish_emiss.in

# 4) Open the analysis notebook (spectra + peel-off images)
jupyter notebook jellyfish_analysis.ipynb
```

The notebook plots the emergent spectrum $J_{\text{out}}(\nu)$, peel-off 2D surface-brightness maps, and a position–velocity diagram from the data cube for both runs.

18. Known Limitations and Assumptions

1. **Neutral fraction.** The converter writes n_H (total hydrogen density). Inside LaRT, the neutral fraction is selected by `par%ionization_model`: a single CIE formula (default), a Voronov+Verner CIE table, $x_{HI} = 1$

(`'full_neutral'`), or a per-cell value from an `xHI` column in the input file (`'from_file'`). For simulations with on-the-fly ionisation, write `xHI` from the RT solver as a passive scalar into the converter, then select `ionization_model = 'from_file'`.

2. **Mean molecular weight.** The temperature calculation in `ramses_read_all_cpus` (used inside the converter) assumes $\bar{\mu} = 1.22$ (fully neutral primordial gas). For ionised or metal-enriched gas this gives an overestimate.

3. **Hydro precision.** The variable `ramses_hydro_prec` in `read_ramses_amr.f90` (default 8) must be changed to 4 for simulations compiled with single-precision hydro.

4. **Non-standard variable ordering.** If the RAMSES simulation includes passive scalars before the thermodynamic variable, the standard assumption (E = var 5) may be wrong. The current reader first checks `hydro_file_descriptor.txt`; if that file is absent, standard RAMSES ordering is assumed.

5. **Octree alignment.** Native RAMSES cell sizes are $\Delta_L = \texttt{boxlen}/(\texttt{nx\_base} \cdot 2^L)$ and are only octree-natural when `nx_base` is a power of 2. For non-power-of-2 base grids the converter automatically re-embeds the populated region in a power-of-2-aligned sub-block and bumps levels accordingly (see Bug 6 above). Generic files produced before this fix are mis-aligned and must be regenerated.

6. **Memory.** All leaf cells are broadcast to every MPI rank in LaRT. For large generic AMR files ($\gtrsim 10^7$ leaf cells) this may require significant RAM per node.

7. **Coordinate convention.** LaRT honors `ORIGINX/Y/Z` headers; the default is centered. If the data does not cover the box center, pole traversal returns $N(HI)_{\rm pole} = 0$ and LaRT prints a clear WARNING before falling back to `tauhomo` normalization. Run `extract_amr_subset.py` (or rebuild from the original snapshot) if the warning appears.

References

[1] Teyssier, R. 2002, A&A, 385, 337 (*RAMSES: a new high-resolution n-body and hydrodynamics code for cosmological simulations*)

[2] Smith, A., Bromm, V., Loeb, A. 2015, MNRAS, 449, 4336 (*Lyman alpha radiative transfer with the LMR scheme*). Equation 35 gives $x_{\rm crit} \simeq (a\tau_0)^{1/3}/5$ used by the per-cell core-skip.

[3] Michel-Dansac, L., Blaizot, J., Garel, T., et al. 2020, A&A, 635, A154 (*RASCAS: RAdiation SCAttering in Astrophysical Simulations*). The per-cell core-skip implementation here follows the RASCAS `module_gas_composition.f90` convention.

[4] Seon, K.-I., Kim, C.-G. 2020, ApJS, 250, 9 (*Lyman-alpha radiative transfer code: LaRT*). Original LaRT description; the volume-averaged $x_{\rm crit}$ prescription retained under `par%core_skip_global=.true.` is the one calibrated here.

[5] Laursen, P., Sommer-Larsen, J., Andersen, A. C. 2009, ApJ, 704, 1640 (*Lyman-alpha radiative transfer with dust: escape fractions from simulated high-redshift galaxies*). Source of the dust prescription used by `par%dust_model='laursen09'`.

[6] Gnat, O., Sternberg, A. 2007, ApJS, 168, 213 (*Time-dependent ionization in radiatively cooling gas*). CIE ion fractions tabulated in `ion_data_mod.f90`.

[7] Asplund, M., Grevesse, N., Sauval, A. J., Scott, P. 2009, ARA&A, 47, 481 (*The chemical composition of the Sun*). Solar abundances tabulated in `ion_data_mod.f90`.